# Épreuve technique de sélection — ADRIA PFE 27

Architecture micro-services, IA et développement full-stack

**Candidate :** Agrigah Aya

**Dépôt :** github.com/Agrigah/Agrigah-Aya-test-adria

**Date :** 30 octobre 2026

# Partie A — Préparation

- Dépôt GitHub (privé pendant l'épreuve, rendu public à la fin) ; projet Maven multi-modules : `discovery-service, config-service, account-service, transaction-service, notification-consumer, chatbot-agent, gateway-service`.
- Commits réguliers toutes les 15 minutes : script `autocommit.sh` (add + commit + push, puis pause de 900 s).
- Google Sheet rempli séparément (hors rapport).

# Partie B — Conception et architecture

Le diagramme de classes reprend le modèle minimal demandé (Account, Transaction) auquel s'ajoutent le type de transaction (DEBIT/CREDIT, nécessaire aux virements), l'événement Kafka et les classes de communication (Feign + Resilience4J).

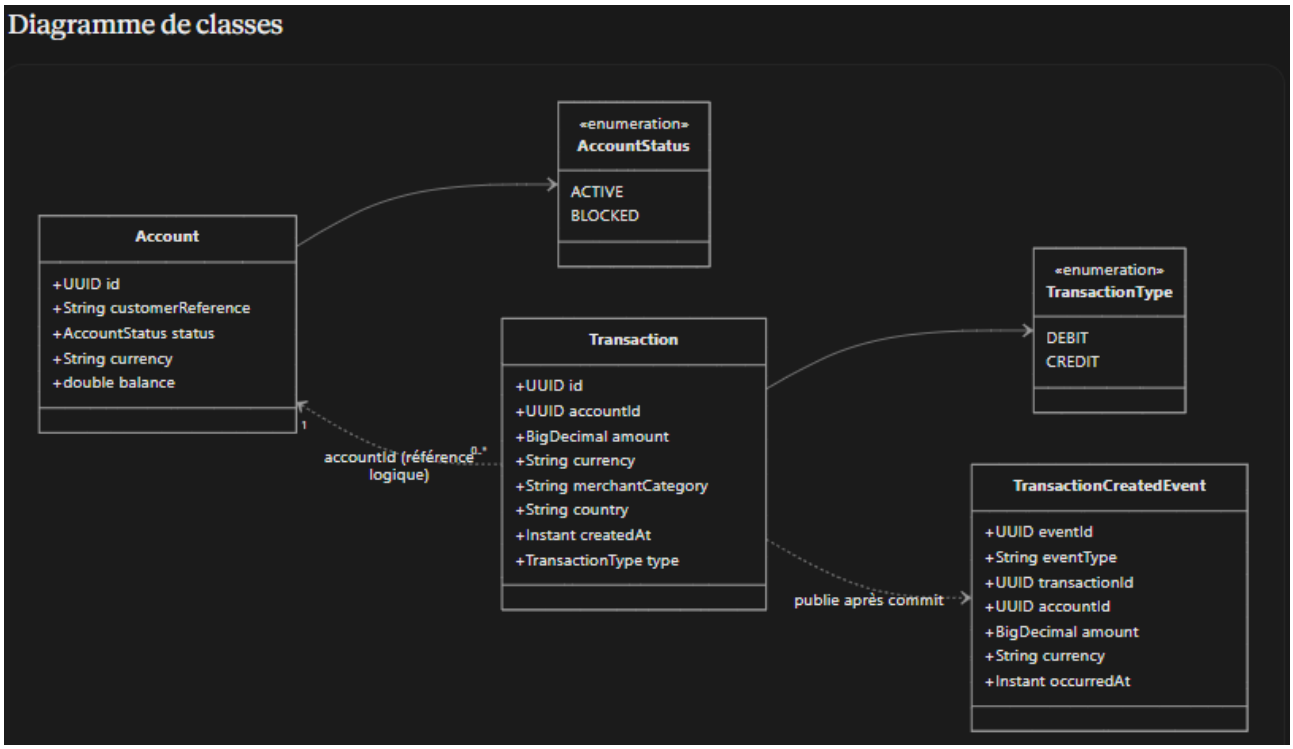



Figure B1 — Diagramme de classes

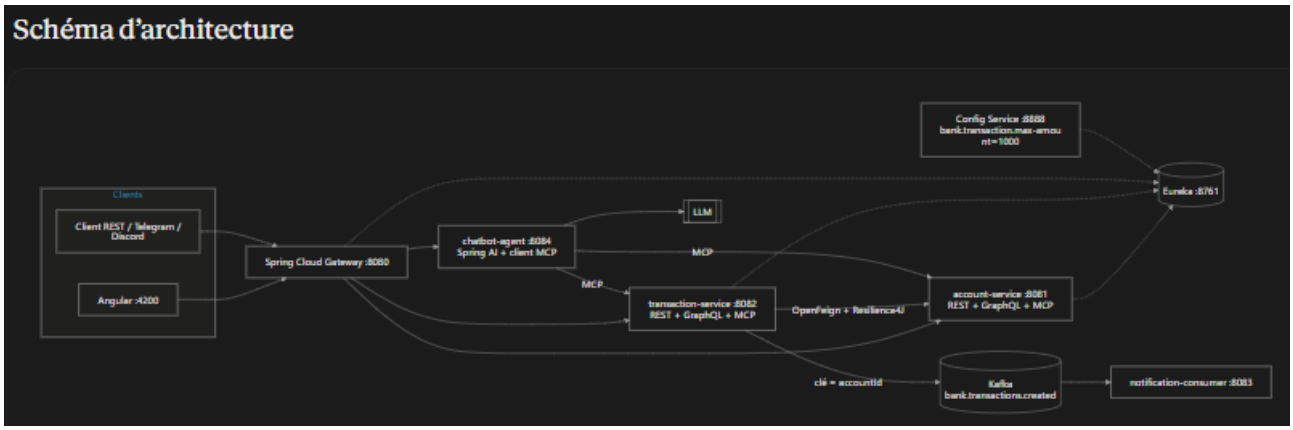



Figure B2 — Schéma d'architecture

**Séparation des couches** (dans chaque micro-service métier) : `domain/` → `repository/` → `service/` ← `web/rest`, `web/graphql`, `mcp/`. Les contrôleurs REST, GraphQL et les outils MCP appellent la **même couche service**.

# Partie C — Micro-services fonctionnels

**account-service (8081)** et **transaction-service (8082)** exposent chacun trois connecteurs : REST (Swagger), GraphQL (GraphiQL) et MCP (SSE).

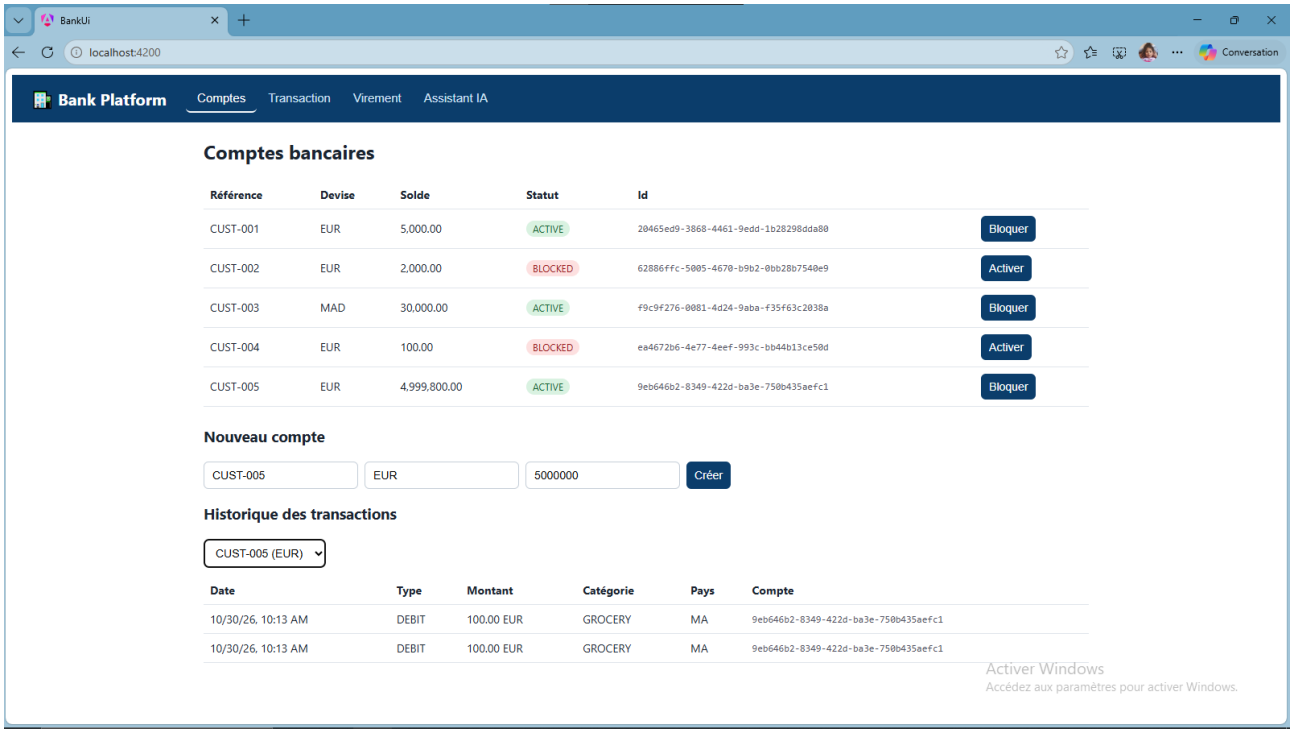
## Règles métier implémentées (transaction-service)

- Montant strictement positif et inférieur ou égal au plafond (`bank.transaction.max-amount=1000`, lu depuis le config-service).
- Le compte doit exister et être **ACTIVE**.
- La devise de la transaction doit être celle du compte.
- Consultation du solde avant chaque transaction (solde suffisant pour un débit), puis mise à jour du solde après enregistrement.
- Virement d'un compte vers un autre : deux écritures liées (DEBIT sur la source, CREDIT sur la destination, catégorie TRANSFER).

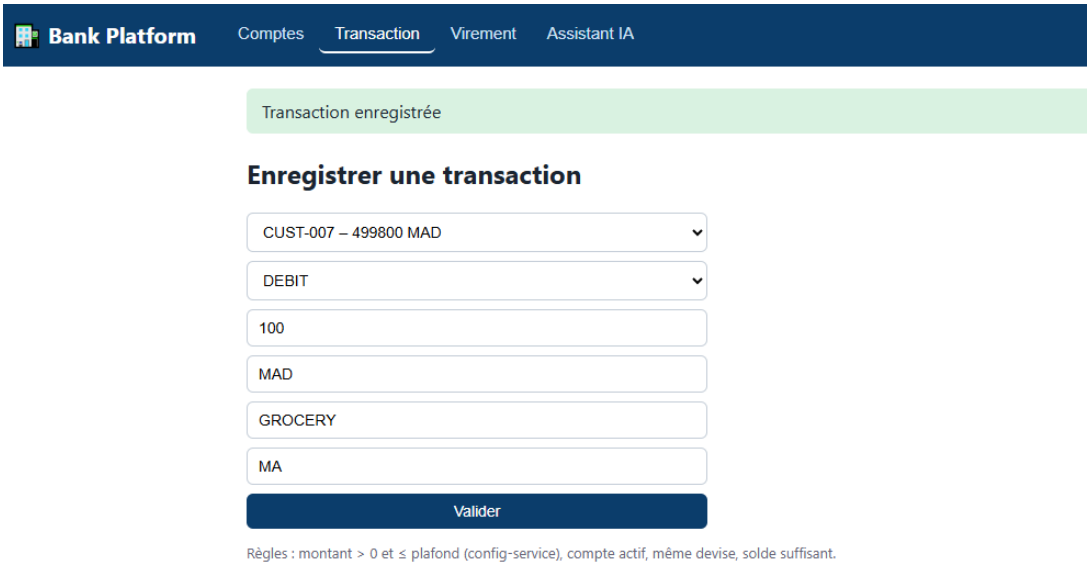
## OpenFeign + Resilience4J

`AccountClient` (Feign) est appelé via `AccountGateway` : **Retry** sur les lectures (erreurs réseau/5xx), **CircuitBreaker** (seuil 50 %, fenêtre de 10 appels, 10 s ouvert) et **fallback** renvoyant 503. Pas de retry sur l'ajustement du solde (opération non idempotente). Les erreurs métier 4xx n'ouvrent pas le circuit.

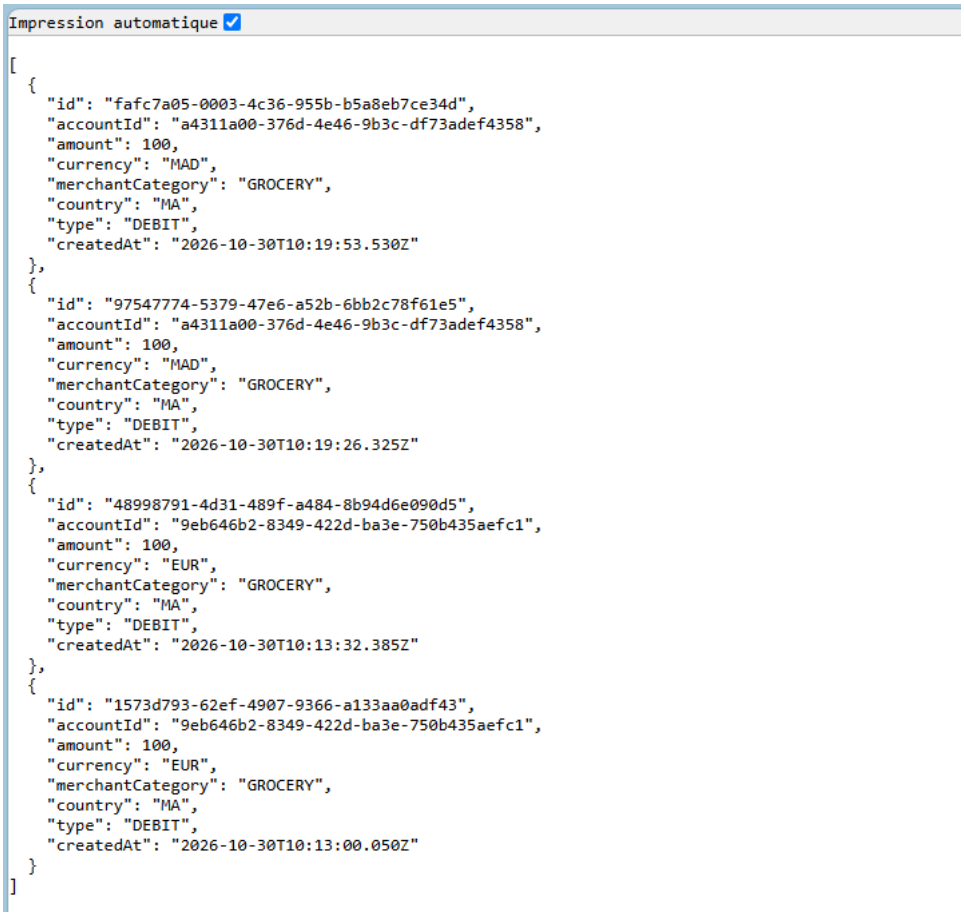
## Tests effectués



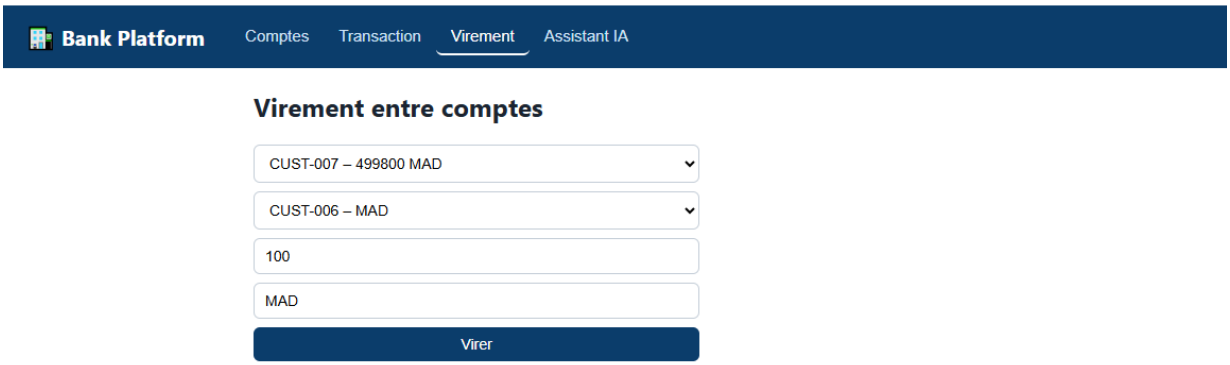

C1 — Liste des comptes (REST account-service via la gateway) : statuts ACTIVE/BLOCKED, création de compte, blocage/activation



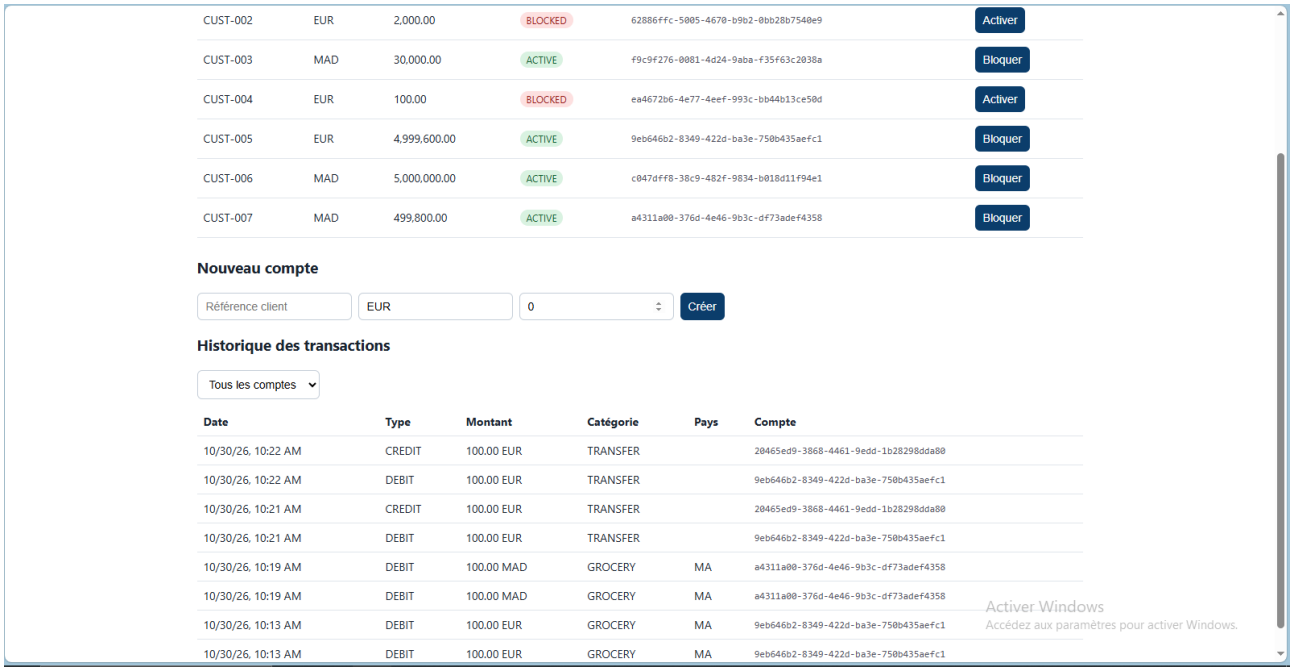

C2 — Enregistrement d'une transaction : « Transaction enregistrée » (règles : montant > 0 et ≤ plafond, compte actif, même devise, solde suffisant)




C3 — Réponse JSON REST de GET /api/transactions (id, accountId, amount, currency, merchantCategory, country, type, createdAt)




C4 — Formulaire de virement entre comptes (CUST-007 → CUST-006, 100 MAD)




C5 — Historique : le virement produit deux écritures TRANSFER (DEBIT sur la source, CREDIT sur la destination) ; les soldes des comptes sont mis à jour

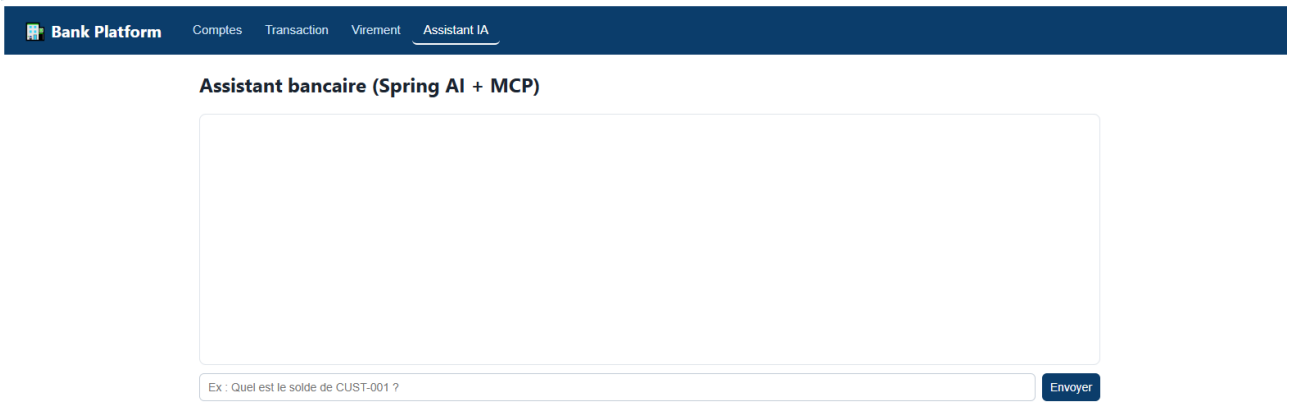
# Partie D — Services techniques

- **Spring Cloud Gateway** (8080) : point d'entrée unique, routage `lb://` via Eureka.
- **Eureka** (8761) : découverte de services.
- **Config Service** (8888) : partage `bank.transaction.max-amount=1000`.

# Partie E — Chatbot agentique

Le module `chatbot-agent` (8084) utilise un **ChatClient Spring AI** relié à un LLM compatible OpenAI et à un **client MCP** connecté aux serveurs MCP d'account-service et de transaction-service ; les outils découverts (consulter/créer/bloquer un compte, enregistrer une transaction, virement) sont fournis au modèle. Une mémoire de conversation (20 messages) est activée. Le prompt système interdit d'inventer

des identifiants ou des soldes.




E1 — Onglet « Assistant IA » de l'interface Angular (appelle POST /api/chat vers le chatbot-agent)

## Partie F — Communication asynchrone avec Kafka

- Après la persistance réussie d'une transaction (publication **après commit**), événement `TransactionCreated` publié dans `bank.transactions.created`.
- Clé du message : `accountId` (ordre garanti par compte).
- `notification-consumer` (8083) consomme le topic et journalise `[EVENT RECEIVED]` sans modifier les données bancaires.

```
{ "eventId":"UUID", "eventType":"TransactionCreated", "transactionId":"UUID",
  "accountId":"UUID", "amount":250.00, "currency":"EUR", "occurredAt":"2026-01-15T10:30:00Z" }
```

## Partie G — Client web Angular

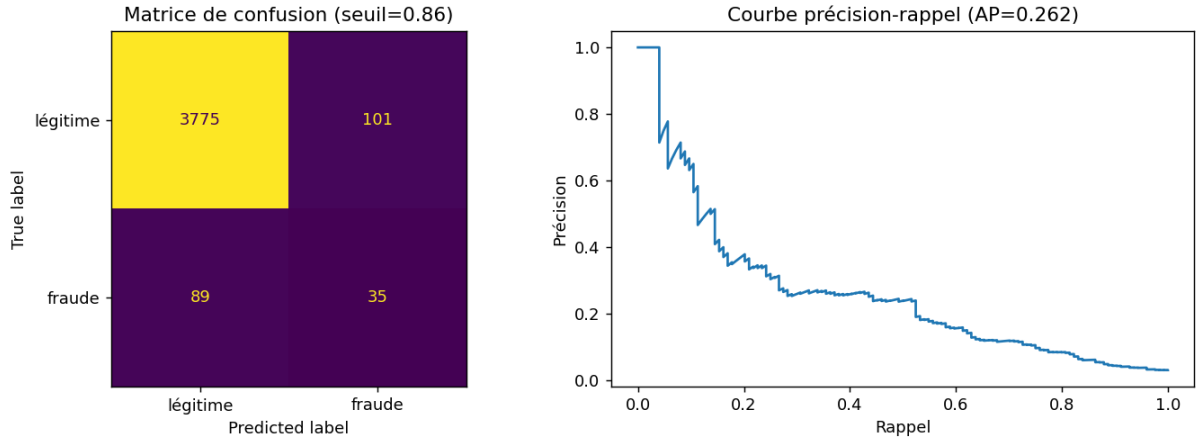
Application Angular `bank-ui` (port 4200) avec quatre onglets : **Comptes** (liste, création, blocage/activation, historique filtrable), **Transaction**, **Virement** et **Assistant IA**. Les captures C1, C2, C4, C5 et E1 ci-dessus illustrent chaque écran.

# Partie H — Détection de fraude (Python)

Notebook : `fraud-detection/fraud_detection.ipynb`. Pipeline scikit-learn : imputation (médiane / valeur la plus fréquente), encodage one-hot des catégories, normalisation des numériques ; identifiants exclus ; séparation 80/20 stratifiée ; prétraitement ajusté **uniquement sur le train** ; modèle choisi par validation croisée stratifiée sur le train ; seuil choisi sur des prédictions hors-échantillon du train (le test sert une seule fois, à l'évaluation).

**Important :** les résultats ci-dessous ont été obtenus sur le **jeu synthétique** généré par `generate_sample_data.py` (20 000 lignes), le fichier officiel `bank_transactions.csv` n'étant pas disponible lors de la rédaction. Les chiffres sont donc indicatifs et doivent être régénérés avec le vrai fichier.

| Indicateur (classe fraude, test) | Valeur |
|---|---|
| Proportion de fraudes | 3,11 % |
| Modèle retenu (AP en CV) | Régression logistique (0,258) |
| Seuil de décision | 0,856 |
| Précision | 0,257 |
| Rappel | 0,282 |
| F1-score | 0,269 |
| Average precision | 0,262 (aléatoire ≈ 0,031) |
| Matrice de confusion [[TN FP],[FN TP]] | [[3775, 101], [89, 35]] |




H1 — Matrice de confusion et courbe précision-rappel (jeu synthétique)

## Questions d'analyse

**1. Pourquoi l'accuracy seule est-elle insuffisante ?** Avec ~3 % de fraudes, prédire « légitime » partout donne ~97 % d'accuracy sans détecter aucune fraude. Il faut regarder précision, rappel, F1 et average precision de la classe fraude.

**2. Compromis faux positifs / fraudes non détectées ?** Un faux positif bloque ou alerte un client honnête (coût d'image et de traitement) ; une fraude non détectée est une perte financière directe. On cherche l'équilibre selon les coûts respectifs.

**3. Effet d'une diminution du seuil ?** Le rappel augmente (plus de fraudes détectées) mais la précision baisse (plus de fausses alertes, donc plus de charge pour les analystes).

**4. Pourquoi une validation temporelle ?** Les fraudes évoluent dans le temps : un découpage aléatoire laisse fuiter le futur dans l'entraînement et surestime les performances. Entraîner sur le passé et tester sur le futur reproduit les conditions réelles.

# Partie I — DevOps

- **Dockerfile unique** pour tous les modules (`--build-arg MODULE=...`), build multi-étapes (Maven puis JRE 21), utilisateur non-root.
- **docker-compose.yml** : Kafka, Eureka, Config, account, transaction, consumer, chatbot, gateway ; `healthcheck` + `depends_on: service_healthy`.
- **GitHub Actions** (`.github/workflows/ci-cd.yml`) : compilation, tests (`mvn verify`), construction d'images (matrice par module), publication sur GHCR, déploiement schématique via SSH + docker compose.

## Réponses courtes

**Secrets vs configuration.** La configuration (ports, URLs, plafond, topics) n'est pas sensible : versionnée, propre à chaque environnement (config-service). Les secrets (clé du LLM, mots de passe, token de registre, clé SSH) ne sont jamais committés ni dans l'image : injectés à l'exécution (`.env` ignoré par git, GitHub Actions secrets, Docker/Kubernetes secrets ou Vault) et renouvelables sans reconstruire l'image.

**Démarrage, disponibilité, vitalité.** Actuator expose `/actuator/health/liveness` (process vivant, sinon redémarrage) et `/actuator/health/readiness` (prêt à recevoir du trafic, dépendances OK) ; une *startup probe* laisse le temps au démarrage. Dans Compose : `healthcheck` et `service_healthy`.

**Logs, métriques, traces.** Logs structurés sur la sortie standard avec identifiant de corrélation, centralisés (ELK/Loki) ; métriques Micrometer/Actuator exposées à Prometheus et visualisées dans Grafana ; traces distribuées via OpenTelemetry/Micrometer Tracing vers Zipkin ou Jaeger.

**Stratégie de rollback.** Images immuables taguées par SHA de commit : revenir en arrière = redéployer le tag précédent (`TAG=<sha précédent> docker compose up -d`). Pour réduire le risque : déploiement progressif (blue/green ou canary) et retour automatique si readiness échoue. Les migrations de base doivent rester rétro-compatibles.




I1 — docker compose ps : Kafka démarré et « healthy » (port 29092 exposé pour les services lancés en local)




I2 — Docker Desktop : projet agrigah-aya-test-adria-pfe-27 en cours d'exécution

## Limites connues

- Keycloak (authentification/autorisations) cité dans l'architecture cible mais **non implémenté**.
- Cohérence solde/transaction assurée par compensation ; un pattern Saga/Outbox serait l'évolution naturelle.
- H2 en mémoire pour la simplicité ; PostgreSQL recommandé en production.